

S06.01 Paris Trees and Gardens worksheet on MongoDB

The context

Trees are a huge asset in the path to resilience with regard to climate change.

The city of Paris has a strong policy to leverage trees in its strategy to lower the temperatures during heat waves.

- "The city has set itself the goal of de-bituminizing 100 hectares and planting 170,000 trees between 2020 and 2026." from [Climate change resilience \(fr\)](#)
- see also "The tree plan" document
<https://cdn.paris.fr/paris/2021/12/13/daf6cce214190a66c7919b34989cf1ed.pdf>



from [this article](#)

You want to launch a new app where people can find shade in the event of a heat wave. Given a user location, the app finds the closest Green Space with the largest amount of trees.

You have an important meeting next week with the Paris team in charge of trees where you want to demonstrate a POC (proof of concept) of your new app.

Fortunately, there is a couple of datasets available on the Paris open data platform that you can leverage

- The [Paris Tree dataset](#) that includes over 210k trees in and around Paris.
- The [Espaces verts](#) dataset (green spaces) with over 2300 areas.

These are real world datasets and as such have their share of anomalies. In this practice, we will use modified versions of these 2 datasets.

In today's workshop you will:

- load both datasets into MongoDB collections
- write aggregation pipelines to explore the datasets and weed out anomalies
- practice creating schema validators
- learn to use [GeoJSON](#)
- identify trees within a greenSpace

Submissions

To submit your answers go to the answers form on <https://forms.gle/iEjXSezcskFr8iQo7>

Fill in your **epita email** and your name

A look at the datasets

You can find the datasets in the github repo

<https://github.com/SkatAI/epita-mongodb/tree/master/data>

Each dataset exists the following versions

- 100 samples: `trees_100.json.zip`, `greenspaces_100.json.zip`
- 1000 samples: `trees_1k.json.zip`, `greenspaces_1k.json.zip`
- full dataset in JSON: `trees.json.zip`, `greenspaces.json.zip`
- full dataset in NDJSON: `trees.ndjson.zip`, `greenspaces.ndjson.zip`

A `json` file contains an array of documents, while a `ndjson` file contains one document per line.

Note: The smaller datasets (100 and 1000 samples) are available if you struggle to import the full datasets. By default, you should try working directly with the full datasets:

Tree sample

Here's a random sample of the trees dataset

```
{
  "idbase": 121837,
  "domain": "Jardin",
  "stage": "Adulte",
  "height": 12,
  "circumference": 290,
  "location": {
    "id_location": "103006",
    "address": "JARDIN DE L HOSPICE DEBROUSSE / 148 RUE DE BAGNOLET",
    "arrondissement": "PARIS 20E ARDRT"
  },
  "geo": {
    "geo_point_2d": "48.86213187971267, 2.406612673774295"
  },
  "taxonomy": {
    "name": "If",
    "species": "baccata",
    "genre": "Taxus"
  }
}
```

Not all the fields are present in that sample. For instance, some trees also have a `taxonomy.variety` field.

Also Note that the geo location value `geo_point_2d` is a string. We will have to convert it into a `Point` data type using an aggregation pipeline

Garden sample

Here's a sample of the garden dataset

```
{
  "nom": "JARDINIERES EVQ DE LA RUE WALDECK-ROUSSEAU",
  "typologie": "Décorations sur la voie publique",
  "categorie": "Jardiniere",
  "nombre_entites": 4.0,
  "id": {
    "id_division": 117.0,
    "id_atelier_horticole": 34.0,
    "ida3d_enb": "JDE13017",
    "site_villes": "SV",
    "id_eqpt": "13017"
  },
  "location": {
    "adresse_numero": 1.0,
    "adresse_type_voie": "RUE",
    "adresse_libelle_voie": "WALDECK ROUSSEAU",
    "code_postal": 75017
  },
  "surface": {
    "superficie_totale_reelle": 45.0,
    "surface_horticole": 45.0
  },
  "history": {
    "annee_de_ouverture": 2024.0
  },
  "geo": {
    "geo_shape": {
      "coordinates": [
        [
          [
            [
              2.286545191756581,
              48.88003700060801
            ],
            // ...
          ]
        ],
        [
          [
            // ...
          ]
        ]
      ],
      "type": "MultiPolygon"
    },
    "geo_point": "48.87986720205693, 2.2864491642985567"
  }
}
```

Visually checking a tree with google maps

As you will see some trees have anomalies in their measurements. The max height is 2524 meters and the max circumference is 80105 cm. This is most probably a human error.

For some dimensions, it is less obvious if the measure is an anomaly or if the tree is really that high or that big.

In that case, it is possible to visually check if a tree has the dimensions indicated in the dataset. Just search for the latitude and longitude (from the `geo_point_2d` field) on google maps and check out the street photos.

For instance this tree is supposed to have `circumference: 1650` (16.5m) but the photo shows only [normal sized Platanes](#).



Setup

All queries and instructions in this document are done in the terminal with `mongosh`.

If you prefer to work with Mongo Compass, directly in ATLAS or using a driver (python, node.js, etc ...) please do so.

This work not about scripting it's about writing MongoDB aggregation pipelines and queries. Feel free to use the best tool available for you.

Local installs

You can work on your local after [installing](#) mongo and mongosh.

To check if `mongoimport` is already installed on your local, do `mongoimport --version`. If it returns the version you're fine, otherwise you must install it.

See this page to download [command line utilities](#). The list of utilities is [here](#). It includes `mongoimport`, `mongoexport`. see also `mongostat` and `mongotop` as diagnostic tools.

ATLAS cluster

You can connect from your local to an ATLAS free-tier cluster.

Atlas free tier clusters offer 512Mb for storage. However, if the current ATLAS free-tier cluster does not have enough space, you should delete it and create a new one that has no data.

Connect

Assuming you've set your MongoDB password as an environment variable `MONGODB_PASSWORD`, you can connect with

```
mongosh "mongodb+srv://<username>:${MONGODB_PASSWORD}@<server_uri>/"
```

Bash

or simply set the whole URI as an environment variable and connect with

```
mongosh ${MONGO_ATLAS_URI}
```

Bash

This connects you to the default `test` database.

Section 1: load the data

Create a new database

Start by creating a new database, let's call it `treedb`.

In MongoDB, to create a new database you simply `use` it

```
use treedb # this creates the database
```

Bash

Here are a few examples of useful MongoDB instructions.

- list databases on the cluster

You can check that the database has been created with `show dbs`

```
Atlas atlas-2914eu-shard-0 [primary] test> show dbs
admin  348.00 KiB
local  8.47 GiB
```

Bash

- Create a `trees` collection with

```
db.createCollection("trees")
```

Bash

- drop a database with `dropDatabase()`

To drop a database, first use it `use sample_mflix` and then you can drop it `db.dropDatabase('sample_mflix')`. You cannot drop a database that is not the current one.

- Empty a collection with `deleteMany()`

```
db.trees.deleteMany({})
```

JavaScript

- delete a collection with `drop()`

```
db.trees.drop()
```

JavaScript

- check the content of a collection with `stats()`

```
db.trees.stats()
```

JavaScript

Here is a good [MongoDB cheatsheet](#) for other instructions.

Let's load the data

A few remarks before you get started:

1. First you need to **download the data from the github into your local and unzip the files**
2. In the following we assume that the data files are in the current folder `./`. Modify with your own path.
3. Since loading the whole dataset may take awhile, it's best to work on a 1k or 100 sample subset. All examples below use the 1k trees subset.

We have several options to load the data from a JSON file. Either in mongosh with `insertMany()` or from the command line with `mongoimport`.

In mongosh with `insertMany()`

The following script

- loads the data from the `trees.json` file
- Insert data into the `trees` collection with `insertMany()`
- times the operation

```
// load the JSON data
const fs = require("fs");
const dataPath = "./trees_1k.json"
const treesData = JSON.parse(fs.readFileSync(dataPath, "utf8"));

// Insert data into the desired collection
let startTime = new Date()
db.trees.insertMany(treesData);
let endTime = new Date()
print(`Operation took ${endTime - startTime} milliseconds`)
```

JavaScript

Using `mongoimport` command line

`mongoimport` is usually the fastest option for large volumes.

By default `mongoimport` takes a ndjson file (one document per line) as input. But you can also use

a JSON file (an array of documents) if you add the flag `--jsonArray`.

The overall syntax for `mongoimport` follows:

```
mongoimport --uri="mongodb+srv://<username>:<password>@<cluster-url>" \
--db <database_name> \
--collection <collection_name> \
--file <path to ndjson file>
```

Bash

Here are other interesting, self explanatory, flags that may come in handy:

- `--mode=[insert|upsert|merge|delete]`
- `--stopOnError`
- `--drop` (drops the collection first)
- `--stopOnError`

In our context, here is a version of the command line, using the `MONGO_ATLAS_URI` environment variable and loading the JSON file `trees_1k.json` in the current folder.

```
time mongoimport --uri="${MONGO_ATLAS_URI}" \
--db treesdb \
--collection trees \
--jsonArray \
--file ./trees_1k.json
```

Bash

which results in

```
2024-12-13T11:41:45.941+0100 connected to: mongodb+srv://[**REDACTED**]@skatai.v
2024-12-13T11:41:48.942+0100 [#####] treesdb.trees 558KB/55
2024-12-13T11:41:52.087+0100 [#####] treesdb.trees 558KB/55
2024-12-13T11:41:52.087+0100 1000 document(s) imported successfully. 0 document(
mongoimport --uri="${MONGO_ATLAS_URI}" --db treesdb --collection trees --fil 0
```

Bash

Q1.1 Write down your import command

Write down your `mongoimport` command as the 1st answer to the 1st question on the quiz

Load the green spaces dataset

Load now the `greenspaces.json` dataset into a new `greenSpaces` collection in the `treesdb` database.

In mongosh:

```
use greenSpaces
```

JavaScript

Then in the terminal:

```
time mongoimport --uri="${MONGO_ATLAS_URI}" \
--db treesdb \
--collection greenSpaces \
--jsonArray \
--file ./greenspaces.json
```

Bash

Check that the import worked with `show collections` and `db.greenSpaces.countDocuments()` which should return a total of 2313 documents.

And now load the entire trees dataset

If you are able to load the 1k sample file, you should now load the entire dataset. This takes a few minutes.

Make sure you first remove all existing documents from the current `treesdb` database with `db.trees.deleteMany({})`.

Then import the whole dataset from the `trees.json` or the `trees.ndjson` file. (unzip first of course)

Note: you don't need to wait for the whole trees dataset to be loaded. You can proceed with the questions while the dataset loads.

Section 2: Data Exploration

In this section, you write aggregation pipelines to explore the trees dataset.

You are asked to write queries for each question below in the answer forms on <https://quiz.skatai.com/quiz/worksheet-on-paris-trees-data-in-mongodb>

Q2.1 Count trees per domain

This is equivalent of the SQL

```
select count(*) as n, domain from trees group by trees order n desc
```

SQL

Write the aggregation pipeline to count trees per domain, order by most common domains first.

Hint: use the `$group` and `$sort` operators.

The most common domain is *Alignments* and the least common one is *DAC* (not clear what *DAC* refers to).. You should obtain 48256 *Alignments* and 33 for *DAC*.

Q2.2 Count the number of trees per stage

The distinct function `db.trees.distinct('stage')` returns 4 values: `[ 'Adulte', 'Jeune (arbre)', 'Jeune (arbre)Adulte', 'Mature' ]`. However, the `stage` field is often missing from the `trees` documents.

Write the aggregation pipeline to count the number of trees per `stage` for all values as well as number of trees with missing stage (null values), order by descending.

Optional: using `$ifNull`, you can specify a string value for null.

```
{
  $ifNull: ["$stage", "MISSING"] // Use "MISSING" as a placeholder for missing '
},
```

JavaScript

Q2.3 Dimensions of trees per stage

Let's calculate some stats on the dimensions of the trees in the database according to their stage.

Group the trees by `stage` and calculate the **count**, **min**, **max** and **average** height and circumference of trees.

- Exclude trees where the stage is missing.
- use `$project` and the `$round` operators to round up the average to 2 decimals
- sort by average height descending

Q2.4 Count remarkable trees

Some trees are flagged with the field `remarkable`. If the field is present it has the value 'OUI', otherwise the field is simply missing.

Write the query that counts how many trees are remarkable using `countDocuments()` and the `$exists` operator. 'OUI'.

Q2.5 What makes a tree remarkable ?

We would assume that remarkable trees are special because of their dimensions. Let's verify that assumption.

Write an aggregation pipeline that compares the average height and circumference of trees that are `remarkable` with ones that are not. What do you conclude, was our assumption correct?

In the aggregation pipeline:

- only consider Platane trees (add `"taxonomy.name" : "Platane"` in the `$match` clause)
- then exclude trees that have zero height or zero circumference
- and exclude trees that are higher than 100m or with a circumference > 500cm
- finally, only consider mature trees (stage = 'Mature')
- don't forget to round up the average height and circumference to 2 decimals. (`$project` and `$round`)

The output should show

- `remarkable` (OUI or `null`)
- `count`
- `avgHeight`
- `avgCircumference`

Q2.6 Top 5 names of trees

What are the most common trees in Paris ?

Write the aggregation pipeline that returns the names of the top 5 most common trees names (`taxonomy.names`) in Paris

Q2.7 Calculate the age of the trees

The age of a tree is highly correlated to its circumference. The relation between age and circumference depends on many factors including the tree species. Here we use a very simplified rule where the `age` is the `circumference` divided by π .

age in years = Circumference in cm / π

We want not only to calculate the age of each tree with but also add that number to a new field `age` to the collection.

2 operators are available for that :

- `$addField`: Only adds new fields, preserves existing ones
- `$setField` (alias `$set`): Can both add new fields AND modify existing ones

However, both operators only work on the current aggregation pipeline. They do not make persistent changes to the collection. For that you need to use the `updateMany()` function instead of an aggregation pipeline.

see [this page](#) and [this one](#) for the documentation and examples on using `updateMany()`

To check that the `age` field has been created, run `db.trees.countDocuments({age : {$exists: true}})`

Q2.8 how many greenSpaces are gardens?

Let's now explore the greenSpaces dataset

We want to check how many of these green spaces are gardens.

Write the aggregation pipeline to count the number of each `categorie` from the greenSpaces collection

Among other categories, we have 19 *Parcs*, 2 *Terrain de boules*, 341 *Murs vegetalises* and 212 *Jardins*.

For the sake of curiosity also check the different `typologie`.

Q2.9 is *perimetre* available for all categories and topologies of greenSpaces ?

We want to see if all categories and typologies of greenSpaces have a perimeter.

Write the aggregation pipeline that counts the number of greenSpaces that have a perimeter per category and typology

Q2.10 Sample 5 greenSpaces at random

Looking at random samples of a collection is often a good way to get acquainted with the data.

Using the `$sample` operator, write the aggregation pipeline that returns 5 randomly selected Green Spaces.

Section 3 : Schema Validation

At this point you have a better understanding of the datasets and you are comfortable writing aggregation pipelines.

The next step is to make sure the data in the database is clean. For that we will create the collection **before** importing the data and specify what sort of data can be added to the collection.

Schema Validation

First please familiarize yourself with schema validation in MongoDB with this document: [Schema validation](#). See this [documentation](#) for a more in depth presentation

We create the collection before importing the data with the function `db.createCollection(name, options)`.

The syntax for create collection can be found [here](#)

Here is a shorter version :

```
db.createCollection( <name>,  
  {  
    validator: <document>,  
  }  
)
```

JavaScript

The **validator** option takes a `document` that specifies the validation rules or expressions.

Check out this example:

```

db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      title: "Student Object Validation",
      required: [ "address", "major", "name", "year" ],
      properties: {
        name: {
          bsonType: "string",
          description: "'name' must be a string and is required"
        },
        year: {
          bsonType: "int",
          minimum: 2017,
          maximum: 3017,
          description: "'year' must be an integer in [ 2017, 3017 ] and is"
        },
        gpa: {
          bsonType: [ "double" ],
          description: "'gpa' must be a double if the field exists"
        }
      }
    }
  }
})

```

from <https://www.mongodb.com/docs/manual/core/schema-validation/specify-json-schema/#create-a-collection-with-validation>

Q3.1 Write a trees schema validator

Your task is to write a validator that requires

- `height < 50`
- `circumference < 500`
- required `taxonomy.name` and `geo.geo_point_2d`
- `validationAction: "error"`

and use it in the `db.createCollection("trees", {<validator>})` function

Note: Before you import the data with `mongoimport` or with `db.trees.insertMany()`, don't forget to empty the collection `db.trees.deleteMany({})` or to drop it `db.trees.drop()` between each run.

Q3.2 Compare validationAction: error and warn

Your task: using `mongoimport` on the 1k sample dataset `trees_1k.json`

- drop the trees collection

- import the dataset with validationAction: "error", and validationAction: "warn".

Write down your observation and conclusion.

Q3.3 Remove geolocation duplicates

Besides crazy heights and absurd circumferences, the dataset also holds geolocation duplicates. Some trees have the same exact latitude and longitude.

Write the aggregation pipeline that finds all the trees that share the same geolocation.

You should find 18 trees (9 pairs) that share the same geolocation if you are working with the the 1k or the whole dataset. (the dataset is designed so that duplicates show as the first documents)

Q3.4 Create a unique index

The best way to avoid duplicates is to add a unique index.

- drop the collection and recreate it with the validator
- add a unique index on the `geo.geo_point_2d` field

```
db.trees.createIndex({ "geo.geo_point_2d": 1 }, { unique: true })
```

JavaScript

Check that index exists with `db.trees.getIndexes()`.

Write down the output of `db.trees.getIndexes()`

Now reimport the trees_1k data

You should see messages like this one during the import.

```
2024-12-08T12:12:32.561+0100    continuing through error: E11000 duplicate key €
```

Bash

Now once you've re imported the data, if you rerun the aggregate pipeline that detects duplicates geolocations, it should return nothing.

Load the whole dataset then take a break

Load the whole dataset

but first:

- drop the `trees` collection
- recreate the `trees` collection with the validator
- create the unique index on `geo.geo_point_2d`
- load the whole dataset with mongoimport using `trees.json` or `trees.ndjson`

This takes around 10 mn. Take a break

At the end you should see:

```
2024-12-08T12:44:27.147+0100 169120 document(s) imported successfully. 1231 docu
```

Bash

Once that is done you should clone the collection so that if anything breaks you do not have to reload the whole dataset

Cloning the collection

You can do so with

```
db.trees.aggregate([
  { $match: {} }, // This matches all documents
  { $out: "trees_backup" } // Creates a new collection named trees_backup
])
```

JavaScript

If you need to restore the `trees_backup` collection, simply drop the `trees` collection and use the same statement.

Section 4: GeoJSON

GeoJSON is a format for encoding geographic data structures. Key points:

- Uses JSON syntax
- Basic types: Point, LineString, Polygon, MultiPoint, MultiLineString, MultiPolygon
- Coordinates in [longitude, latitude] order
- Example Point:

```
{
  "type": "Point",
  "coordinates": [2.3522, 48.8566] // Paris coordinates
}
```

JavaScript

geoJSON is used widely in mapping applications and spatial databases like MongoDB

see [geoJSON in MongoDB](#) for reference

Note: contrary to common usage where latitude comes before longitude, geoJSON points have the longitude first and the latitude second

Once we have a geoJSON point for each tree, we can find all the trees within 100 meters of the Eiffel tower with the `$near` operator.

```
db.trees.find({
  location: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [2.294694, 48.858093] // Eiffel Tower coordinates
      },
      $maxDistance: 100 // in meters
    }
  }
})
```

We can also use the `$geoNear` operator:

```
db.trees.aggregate([
  {
    $geoNear: {
      near: {
        type: "Point",
        coordinates: [2.294694, 48.858093]
      },
      distanceField: "distance",
      spherical: true,
      query: { "genre": "Tilia" } // Only Linden trees
    }
  }
])
```

The `geo.geo_point_2d` field in the database is just a string with `<latitude>,<longitude>`. Let's convert it to GeoJSON format. This will allow us to find the trees near a specific geolocation.

In this section, you will

- convert string to geojson
- find your address lat and long
- given location find number nearest trees
- do you have a remarkable tree in the vicinity?

Q4.1 Convert the `geo_point_2d` string into an array of strings

We want to convert the `geo_point_2d` string `<lat>, <long>` into a geoJSON point. Start by converting the string into an array of floats, and swap the coordinates.

Write an aggregation pipeline that splits the `geo_point_2d` field into an array of 2 numbers while swapping the coordinates.

Hint: use the following operators

- `$addFields` to create a new field
- `$split` to split the string with respect to the comma `,`
- `$convert` to convert a string to double (float)
- `arrayElemAt(array, n)` to get the nth element from an array

You can also use the `$let` operator to create temporary variables within an expression.

```
$let: {
  vars: { /* define variables here */ },
  in: /* use variables here with $$varname */
}
```

JavaScript

Q4.2 Convert the array of floats into a Point

Let's expand on that pipeline and now cast the array of floats into a Point.

Make sure

- the new field is nested in the `geo` field
- name the new field : `geo_point`

Write the aggregation pipeline that will include the new field in the `geo` field as such:

```
{
  geo: {
    geo_point_2d: '48.837794432065046, 2.3777788022073083',
    geo_point: {
      type: 'Point',
      coordinates: [ 2.3777788022073083, 48.837794432065046 ]
    }
  }
}
```

JavaScript

Q4.3 Make the new field permanent

If you've used `$addFields`, the new field is created on the fly and not saved as permanent in the collection. (`db.trees.findOne()` won't return the new field).

Write the statement using `$set` and `$updateMany()` that actually adds the new field into the collection.

check the geo_point field

Check that everything is as expected. The `db.trees.findOne()` command should return something similar to

```
{
  _id: ObjectId('675ea31285f935714045b298'),
  idbase: 2041515,
  domain: 'Jardin',
  stage: 'Jeune (arbre)',
  height: 5,
  circumference: 50,
  location: {
    id_location: '101002',
    address: 'SQUARE ERNEST GOUIN / 19 RUE EMILE LEVEL',
    arrondissement: 'PARIS 17E ARRD'T'
  },
  geo: {
    geo_point_2d: '48.89391877343201, 2.317941722152433',
    geo_point: {
      type: 'Point',
      coordinates: [ 2.317941722152433, 48.89391877343201 ]
    }
  },
  taxonomy: { name: 'Saulle', species: 'sp.', genre: 'Salix' }
}
```

Q4.4 convert gardens geolocation

The greenSpaces collection also has a geo_point as a string of `<lat>, <long>`.

Write the updateMany() statement that will transform the greenSpaces.geo.geo_point into a geoJSON type.

Add a geo spatial index

before we can run geo queries we need to add a geo spatial index

```
db.trees.createIndex({ "geo.geo_point": "2dsphere" })
db.greenSpaces.createIndex({ "geo.geo_point": "2dsphere" })
```

Find the number of trees in the vicinity of an address

Now let's take an address

You can find the related longitude and latitude using google maps. just click right on the red pin

You get for instance : 48.82124804336634, 2.363645912660704

Let's find the number of trees around this address, in a radius of 50m

JavaScript

```
// Find how many trees are within 100 meters of Tricotin
db.trees.find({
  location_geojson: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [2.363645912660704, 48.82124804336634] // Tricotin coordir
      },
      $maxDistance: 100 // in meters
    }
  }
}).count()
```

What about Epita ? with coordinates 2.362806368481762, 48.815820795886815

JavaScript

```
// Find how many trees are within 100 meters of Epita
db.trees.find({
  location_geojson: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [2.362806368481762, 48.815820795886815] // Epita coordinat
      },
      $maxDistance: 100 // in meters
    }
  }
}).count()
```

only 11 trees!

What about your own address ?

Q4.5 Most common trees in an area

Write the aggregation pipeline that returns the top 5 most common tree name in the vicinity of a particular garden

You can set the greenspace name as a variable

For instance:

JavaScript

```
let greenspace_name = 'SQUARE EMILY DICKINSON'
let radius = 100
```

You need to write one query and one pipeline

- the query finds the greenSpace with the name

- the pipeline uses `$geoNear` on the trees collection with the retrieved point.

Q4.6 find if a tree belongs to a garden

- Find the greenSpace with categorie 'Jardin' an typologie 'Jardins privatifs' and the largest surface_calculée.
 - store the document in a variable (jardin)
- Create a 2dsphere index on greenSpaces.geo.geo_shape
- check that the index has been created
- Write the pipeline to find how many trees are in this *Jardin*
 - using the `$geoIntersects` operator

```
let jardin = db.greenSpaces.aggregate([
  {
    // Step 1: Filter green spaces with the 'Jardin' category
    $match: {
      categorie: "Jardin",
      typologie: "Jardins privatifs"
    }
  },
  {
    // Step 2: Sort by 'surface_calculée' in descending order
    $sort: {
      "surface.surface_calculée": -1
    }
  },
  {
    $project: {
      nom: 1,
      typologie: 1,
      categorie: 1,
      surface :1 ,
      geo: 1,
    }
  },
  {
    // Step 3: Limit to the top result (the largest surface_calculée)
    $limit: 1
  }
]);
```

JavaScript

```
const geoShape = jardin.geo.geo_shape;

const treesInShape = db.trees.find({
  "geo.geo_point": {
    $geoWithin: {
      $geometry: geoShape
    }
  }
});
```

Q4.7 add a `tree_count` field for all green spaces

Now write the `updateMany()` statement to add a `tree_count` new field to all the green spaces documents where `tree_count` counts the number of trees from the trees collection that are in the green space geo shape

Q4.8 Include trees in the greenSpaces collection

In the greenSpaces collection, add the list of trees in the garden as an array.

Only list the height, circumference, and taxonomy (name, species, genre, variety) of the tree.

Conclusion

Now, given a green space, we can easily list all the trees and their dimensions in that green space.

You can more easily build an app on top of that collection to display trees on any Paris Green Space. And since you have the geoJSON geo location of each green space, a user can find the closest place with the highest number of trees to get some rest in the case of a *Canicule*.